U24AI006

ASSIGNMENT -03

1. Implement a Simple Reflex Agent for a vacuum cleaner that can navigate a three-room environment (A, B, and C) and maintain cleanliness. Identify the “rationality” of its actions. How will you define the performance cost? Define the complete Rule Table and store it either in a dictionary/vector/matrix based on the language you implement the vacuum cleaner. Show the simulation output with percept, action, and location. Do you need priorities in the rules?

2. Design a Simple Reflex Agent for an Indian Railways Level Crossing. The agent must ensure maximum safety for road traffic while minimizing delays for the train, using multi-source sensory input without historical tracking.

   The environment consists of a double-track railway line intersecting a busy state highway. The **percepts** include the following:

   - Track Sensors (Inbound/Outbound): Sensors placed 2km away that detect if a train is currently passing over them. It is either detected or not detected.
   - Obstacle Sensors (Yellow Box): A grid of sensors on the road between the gates to detect “stuck” vehicles (e.g., an auto-rickshaw or buffalo).
   - Manual Emergency Input: A physical lever for the station master. It is either Neutral or Active.

   Actuators:

   - Gate Arm: Lower/Raise
   - Hooter/Siren: On/Off
   - Signal to Train: Green (Safe to proceed)/Red (Emergency Stop)

Design the rules set. Show the simulation output with percept, action, and location. Do you need priorities in the rules?

Code:

```
import random

class VacuumEnvironment:
    def __init__(self):
        self.location_states = {
            'A': 1,
            'B': 1,
            'C': 1
        }
        self.agent_location = 'A'
        self.performance_score = 0

    def get_percept(self):
        status = 'Dirty' if self.location_states[self.agent_location] == 1 else
'Clean'
        return (self.agent_location, status)
```

```
    def execute_action(self, action):
        cost = 0

        if action == 'Suck':
            if self.location_states[self.agent_location] == 1:
                self.location_states[self.agent_location] = 0
                cost = 10
                print(f"  Action Effect: Room {self.agent_location} is now CLEAN.")
            else:
                cost = -1
                print("  Action Effect: Wasted energy (Room was already clean).")

        elif action == 'Right':
            if self.agent_location == 'A':
                self.agent_location = 'B'
                cost = -1
            elif self.agent_location == 'B':
                self.agent_location = 'C'
                cost = -1
            else:
                cost = -1
            print(f"  Action Effect: Moved Right to {self.agent_location}.")

        elif action == 'Left':
            if self.agent_location == 'C':
                self.agent_location = 'B'
                cost = -1
            elif self.agent_location == 'B':
                self.agent_location = 'A'
                cost = -1
            else:
                cost = -1
            print(f"  Action Effect: Moved Left to {self.agent_location}.")

        elif action == 'NoOp':
            cost = 0

        self.performance_score += cost
        return cost

class SimpleReflexAgent:
    def __init__(self):
        self.rules = {
            ('A', 'Dirty'): 'Suck',
```

```
            ('A', 'Clean'): 'Right',
            ('B', 'Dirty'): 'Suck',
            ('B', 'Clean'): 'Left',
            ('C', 'Dirty'): 'Suck',
            ('C', 'Clean'): 'Left'
        }

    def select_action(self, percept):
        location, status = percept
        if (location, status) in self.rules:
            return self.rules[(location, status)]
        else:
            return 'NoOp'

def run_simulation(steps=10):
    print("--- Starting Vacuum World Simulation (3 Rooms) ---")

    env = VacuumEnvironment()
    agent = SimpleReflexAgent()

    print("\n[Agent's Internal Rule Table]")
    print(f"{'Percept (Loc, Status)':<25} | {'Action':<10}")
    print("-" * 40)
    for percept, action in agent.rules.items():
        print(f"{str(percept):<25} | {action:<10}")
    print("-" * 40 + "\n")

    for step in range(1, steps + 1):
        print(f"Step {step}:")
        percept = env.get_percept()
        print(f"  Percept:  Agent sees {percept}")
        action = agent.select_action(percept)
        print(f"  Action:   Agent decides to '{action}'")
        step_cost = env.execute_action(action)
        print(f"  Score Change: {step_cost} | Total Performance:
{env.performance_score}")
        print("-" * 30)

    print("\n--- Simulation Complete ---")
    print(f"Final Performance Score: {env.performance_score}")

run_simulation(steps=8)
```

:

OUTPUT:

```
QUESTION 1.py"
--- Starting Vacuum World Simulation (3 Rooms) ---

[Agent's Internal Rule Table]
Percept (Loc, Status)     | Action
----------------------------------------
('A', 'Dirty')            | Suck
('A', 'Clean')            | Right
('B', 'Dirty')            | Suck
('B', 'Clean')            | Left
('C', 'Dirty')            | Suck
('C', 'Clean')            | Left
----------------------------------------

Step 1:
  Percept:  Agent sees ('A', 'Dirty')
  Action:   Agent decides to 'Suck'
  Action Effect: Room A is now CLEAN.
  Score Change: 10 | Total Performance: 10
------------------------------
Step 2:
  Percept:  Agent sees ('A', 'Clean')
  Action:   Agent decides to 'Right'
  Action Effect: Moved Right to B.
  Score Change: -1 | Total Performance: 9
------------------------------
Step 3:
  Percept:  Agent sees ('B', 'Dirty')
  Action:   Agent decides to 'Suck'
  Action Effect: Room B is now CLEAN.
  Score Change: 10 | Total Performance: 19
------------------------------
Step 4:
  Percept:  Agent sees ('B', 'Clean')
  Action:   Agent decides to 'Left'
  Action Effect: Moved Left to A.
  Score Change: -1 | Total Performance: 18
------------------------------
Step 5:
  Percept:  Agent sees ('A', 'Clean')
  Action:   Agent decides to 'Right'
  Action Effect: Moved Right to B.
  Score Change: -1 | Total Performance: 17
------------------------------
Step 6:
  Percept:  Agent sees ('B', 'Clean')
  Action:   Agent decides to 'Left'
  Action Effect: Moved Left to A.
  Score Change: -1 | Total Performance: 16
------------------------------
```

U24AI006

CODE(2):

```
scenarios = [
    [0, 0, 0],   # Case 0: Everything is normal
    [1, 0, 0],   # Case 1: Train detected (Lower Gate)
    [1, 1, 0],   # Case 2: Train detected BUT track is blocked (Safety First!)
    [0, 0, 1]    # Case 3: Manual Emergency Lever pulled
]

class RailwayAgent:
    def decide(self, percept):
        train, obstacle, emergency = percept

        if emergency == 1 or obstacle == 1:
            return ["RAISE", "RED", "ON"]

        if train == 1:
            return ["LOWER", "GREEN", "ON"]

        return ["RAISE", "GREEN", "OFF"]

agent = RailwayAgent()

print("--- INDIAN RAILWAYS LEVEL CROSSING SIMULATION ---")
print(f"{'Percept (T,O,E)':<18} | {'Gate':<8} | {'Signal':<8} | {'Hooter'}")
print("-" * 60)

for p in scenarios:
    act = agent.decide(p)
    print(f"{str(p):<18} | {act[0]:<8} | {act[1]:<8} | {act[2]}")
```

Output:

```
QUESTION 2.py"
--- INDIAN RAILWAYS LEVEL CROSSING SIMULATION ---
Percept (T,O,E)    | Gate     | Signal   | Hooter
------------------------------------------------------------
[0, 0, 0]          | RAISE    | GREEN    | OFF
[1, 0, 0]          | LOWER    | GREEN    | ON
[1, 1, 0]          | RAISE    | RED      | ON
[0, 0, 1]          | RAISE    | RED      | ON
```